Assignment 4: Inverse Problems

Practical information

Deadline: Saturday 28/2 23.59

Resources:

- ERDA for file storage
- Jupyter for the Terminal to access MODI
- Benchmarks through SLURM on MODI (one node!)

Handin:

- Total assignment: a report of up to 3 pages in length (excluding the code)
- Your seismogram_omp code – all versions uploaded separately.
- An up to 3-minute video where you scroll through the code and explain (see Task 1)
- Use the template on Absalon to include your codes in the report

This assignment has to be handed in individually

Introduction

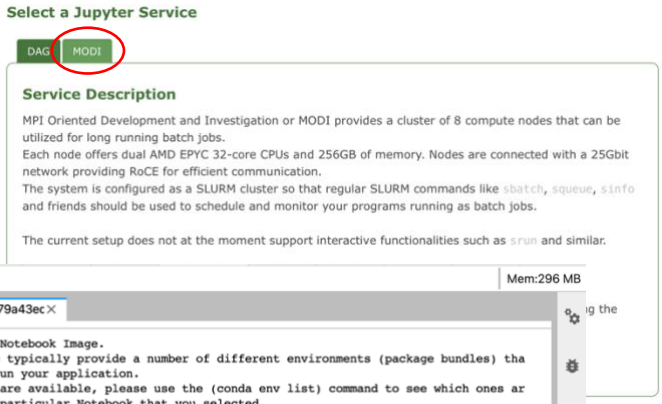
Inverse problems use laboratory or field data to infer information about the properties or internal structure of an object. To solve an inverse problem, we need not only the data but also a relation between data and model parameters that represent the structure of the object. This relation can be an explicit function that maps parameters to data but is more often an algorithm represented by a computer code.

Inverse problems where the mathematical relation is available are often linear, and this allows us to directly solve the problem through matrix algebra, which is well-understood and relatively fast. In many other cases, however, data is related non-linearly to model parameters. This is the case in, e.g., wave propagation problems where waveforms (seismic, electromagnetic, etc.) are used for reconstruction of objects through which the waves have propagated. Since we may not have direct access to the mathematical structure of the problem in such cases, we can only base our solution to the inverse problem on computational strategies that search for reasonable values of model parameters fitting the data within its observational uncertainty. The algorithmic search for solutions consists of two components: (1) a method to compute the data-fit from any set of model parameters, and (2) a strategy for proposing the next set of model parameters to be tested. The first component can be very time consuming, as it is often based on a simulation of the physical process considered in the problem.

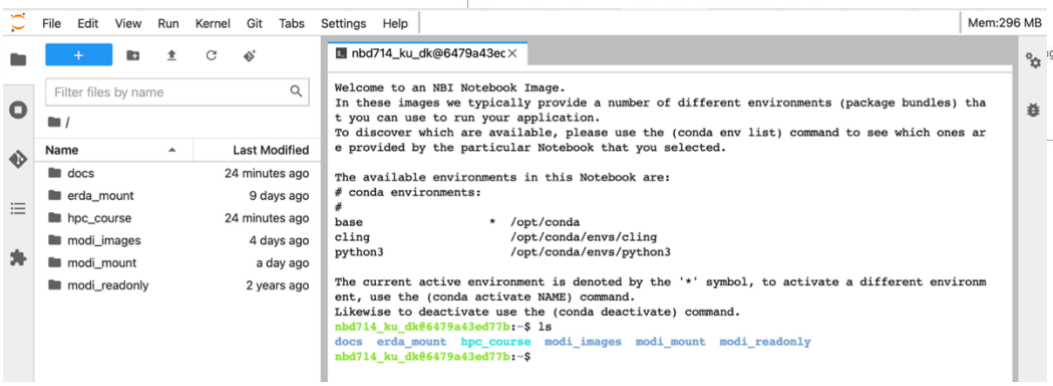
We shall here look at a simple wave propagation problem where plane waves propagate through a medium of plane-parallel layers. The task is to simulate the wave propagation as fast as possible, thereby enabling a more efficient search for solutions to the problem. The problem can be "linearized", but the solution to the resulting linear problem may deviate from the correct non-linear solution. We will look at the physical difference between the complete and the linearized formulation and try to understand why the fully non-linear formulation is better, but so much more computationally demanding.

MODI

For this assignment we need g++ to compile and MODI for benchmarking. You can read more about MODI in the user guide: https://erda.dk/public/MODI-user-guide.pdf



Spin up a Jupyter session on MODI selecting the "HPC notebook" notebook image. In the



terminal (or the folder view on the right side) you can see a number of folders.

The different folders contain:
`docs`: the MODI user guide.
`erda_mount`: your own files.
`hpc_course`: course files.
`modi_images`: images of virtual machines that can be used when submitting jobs.
`modi_mount`: this folder is the only one that can be seen from the cluster nodes. You need to copy any executables you use (this is called *staging*) to this filesystem before submitting a job.
`modi_readonly`: this folder can be ignored.

PREPARATIONS

Start by copying the exercise to your storage area and enter into the folder. You can write '`ls`' to get a file listing of the folder.

```
cd erda_mount/HPPC
cp -a ~/hpc_course/module4 .
cd module4
ls
```

To be able to edit the files for the exercise navigate to the same folder in the file view. Here you can see eight files:

- `Makefile`
- `seismogram_omp.cpp`
- `seismogram_seq.cpp`
- `job.sh`
- `job_weak_scaling.sh`
- `velocity_data.txt`
- `density_data.txt`
- `wave_data.txt`

Before you can run the code, you need to compile it. This can be done by running `make` in the terminal.

For the first task, one binary is produced: `mp`. The `seismogram_seq.cpp` code is there to give you a backup, a corresponding binary, `seq`, is produced. To run the code on the login machine of the MODI cluster using e.g. 4 threads you can do:

```
$ env OMP_NUM_THREADS=4 ./mp
```

Code structure

The code has two parts. It contains a main program which reads in the data files. From the main routine the `propagator` routine is called. The routine calculates the seismogram which is expected given the density profile, the wave-speed profile and the underground wave. To calculate the seismogram we need to go from real space to Fourier space, and therefore the program also contains two simple functions `fft` and `ifft` that uses divide and conquer to compute the (inverse) FFT.

You can change the workload in the program by changing the number of frequencies computed. This is controlled through a preprocessor macro in the Makefile. In the code is written:

```
// The number of frequencies sets the cost of the problem
#ifndef NFREQ
#define NFREQ (64*1024)
#endif
const long nfreq = NFREQ; // frequencies in spectrum
```

And this can be controlled during compilation with the option "-D" to the compiler, such as

```
g++   seismogram_omp.cpp -O3 -ffast-math -Wall -march=native -g -
std=c++14 -DNFREQ=16384 -DNUMA_ALLOCATOR= -fopenmp -o mp
```

which can further be specified when doing "make" on the command line like this

```
$ make NFREQ=16384
```

Because of the very simple FFT algorithm, `nfreq` must be a power of two.

At the top of the code, a template class called `NUMA_Allocator` has been included. It allows to allocate a vector or array of elements that are spread out in memory according to the scheduling defined by the OpenMP pragma "`pragma omp parallel for schedule(static)`" on line 53. It is enabled again using a preprocessor macro called `NUMA_ALLOCATOR` by changing the typedef statements for `ComplexVector` and `DoubleVector` around line 85.

OpenMP thread checker

In the lecture we discussed the concept of a thread checker. You can access the thread checker by changing the compilation flags. Open the Makefile and comment in the relevant compile options. Running with the thread checker options, assuming you have bugs, you will see a lot of output scrolling past on the screen. This can become too much, and it is better to redirect the screen output to a text file, like

```
$ env OMP_NUM_THREADS=4 ./mp >& out
```

The file can afterwards be inspected in the editor. If you want a summary, you can use the `grep` command to filter the output like this

```
$ grep SUMMARY out
```

In the terminal, or you can search for "SUMMARY" in the editor. This can also be done in one go

```
$ env OMP_NUM_THREADS=4 ./mp |& grep SUMMARY
```

Once you have removed the bugs, remember to change back to the fast compile settings with -O3, make clean, and make again, before you do benchmarks. A thread checker intercepts all memory accesses

checking for race conditions or undefined behaviour and can slowdown the program by a factor of 10 to 100. The thread checker available on MODI is unfortunately not very advanced. It will believe it has found problems, which are not really there. These are called *false positives*. You should therefore take the list of *possible* race condition with a grain of salt and evaluate them one by one, convincing yourself that there is no problem.

We *strongly* recommend using the thread sanitizer if you develop an OpenMP program with just a small level of complication; it *will* at some point find threading errors.

Task 1: OpenMP parallelise the program (points 5)

Use OpenMP pragmas to parallelise the code. It is up to you, which strategies you would like to use. For each different strategy to parallelise the code you can get a point. To get the point, besides your implementations submitted as code and in pdf, you should also include a table in your report where you list the different code versions with a one-line description of how the differ from last version or what they do (e.g. "fused parallel regions").
In your video you must explain briefly each version, while you show the code in an editor, and why you expect the version to be an improvement of the last version (even if they do not improve performance and/or scaling).
Examples of different strategies could be replacing many parallel regions with a single region, removing explicit or implicit barriers, employing new types of pragmas, or using the pragmas in a different manner.
Remember always to test that you get the same checksum and include a line about it in the report, below the table.

Task 2: Weak scaling using SLURM (points 5)

When benchmarking the performance of your program, use the MODI servers. To get exclusive access to a machine you need to submit your run through SLURM. An example of a SLURM script, job_scaling.sh, that runs the parallel program on a varying number of cores five times is included. The "exclusive" flag means that there will only be one user on the nodes. "modi_HPPC" indicates the queue. Each node has 64 cores, so this is highly wasteful way to run, but it is a good way to get reliable benchmarks. The apptainer image is needed because each notebook image has a different set of software installed. To complete the task you have to

1. Make weak scaling plots (1 point): You should provide experimental results of running using 1, 2, 4, 8, 16, 32, and 64 threads. The results should be presented in three easy-to-read figures, which show the wall clock time of (i) the program excluding the FFT (ii) only the FFT, and (iii) running the whole program. In each figure there should be one line per code version. Run each benchmark experiment at least 5 times to get an error bar on the measurements. Choose `nfreq` high enough (f.x. `65536*ncore`) such that when going from 1 thread to 2 threads the wall clock is almost the same. E.g. the speedup is close to 2.
2. Compute parallel fractions (1 point): Measure the parallel fraction for each of the different strategies by fitting the data for cases (i) and (ii) to an appropriate scaling law. Notice that in your plot show wall clock time versus number of threads. You need to transform this into speedup as a function of threads taking in to account the number of threads and the workload as discussed in class. The FFT routine is not linear in cost as a function of `nfreq`, but rather the cost of the FFT scales logarithmic as O(N logN) for N points. Please write how you account for this in your scaling laws.
3. Discuss and analyse (3 points): Interpret and discuss your results for each of your different strategies. Do they behave as you expected? You could interpret your results considering the parallelization strategies involved, the code, and the shared memory architecture of ERDA.

Bonus Task – deep dive: variable workloads and high thread counts

Bonus task 1: You can consider redoing the analysis of Task 2 using a finer grid of threads. Then the workload will be quite variable, and this must be considered when doing the scaling and computing the workload.

Bonus task 2: Try running your code with and without hyperthreading; does it make a difference, at what scales, and do you understand why? What about Numa allocator vs not using NUM allocator; how many cores should be needed – at least – before that makes a difference?

You cannot get any points for this bonus task, but it will give you a lot of insight in to running OpenMP at scale and pushing the hardware to its limits. You are most welcome to ask questions about it, and approach us in week 5 for feedback.